

TDMA Schedule Planner and Spatial Reuse Optimizer

Comprehensive Technical Design Document & Verification Report — v1.0

Author: Athish M | Repository: tdma-schedule-optimizer

1. Executive Summary & Problem Understanding

In shared-spectrum radio frequency (RF) networks, transceivers share a common wireless channel. Uncontrolled concurrent transmissions cause destructive interference and packet loss. **Time Division Multiple Access (TDMA)** resolves collisions by dividing time into repeating frames composed of discrete, equal-duration timeslots. Nodes are allocated slots granting exclusive transmission rights.

The system coordinates static transceivers at 2D Euclidean coordinates with radio range $R = 500.0\text{ m}$. The schedule must resolve three fundamental medium access conditions:

- **Distance-1 Conflict (Direct In-Range):** Nodes with physical distance $d(u, v) \leq 500.0\text{ m}$ cannot transmit in the same slot due to mutual receiver saturation and packet collisions.
- **Distance-2 Conflict (Hidden Terminal):** Nodes that are out of direct range but share a common neighbor cannot share a slot; simultaneous transmissions would collide at the shared intermediate receiver.
- **Spatial Concurrency (Reuse):** Nodes separated by $\geq 3\text{ hops}$ (or in disconnected components) can safely reuse the same slot without mutual interference, maximizing aggregate spectral efficiency.

Global Optimization Goal: Minimize the frame length (number of distinct timeslots K). Minimizing K maximizes per-node channel throughput and minimizes latency.

2. Mathematical Modeling & Graph Theory

Let $V = \{v_1, v_2, \dots, v_n\}$ be the radio nodes with coordinates $p(v_i) = (x_i, y_i)$.

- **Physical Connectivity Graph $G = (V, E)$:** Undirected edge $(u, v) \in E$ exists iff Euclidean distance $d(u, v) \leq 500.0\text{ m}$ (evaluated with 10^{-9} m numerical tolerance).
- **Conflict Graph $G_{\text{conflict}} = G^2$:** Graph square of G where an edge connects any node pair within 2 hops in G ($1 \leq \text{dist}_G(u, v) \leq 2$).

2.1 Proof of Equivalence: Distance-2 Coloring of $G \equiv$ Vertex Coloring of G^2

Theorem: A timeslot assignment $c: V \rightarrow \{0, \dots, K-1\}$ is conflict-free iff c is a valid vertex coloring of G^2 .

Proof: ($\Rightarrow$) Suppose c is conflict-free. If $(u, v) \in E(G^2)$, then $\text{dist}_G(u, v) \in \{1, 2\}$. If distance is 1 (adjacent), direct collision rules require $c(u) \neq c(v)$. If distance is 2 (shared neighbor), hidden terminal rules require $c(u) \neq c(v)$. Thus no adjacent vertices in G^2 share a color.

($\Leftarrow$) Suppose c is a valid vertex coloring of G^2 . For any pair with $\text{dist}_G(u, v) \leq 2$, $(u, v) \in E(G^2)$, guaranteeing distinct slots. If $\text{dist}_G(u, v) \geq 3$, $(u, v) \notin E(G^2)$, allowing valid spatial reuse. [Q.E.D.]

Complexity & Bounds: Determining the minimum frame length is NP-hard. It is bounded by $\omega(G^2) \leq \chi(G^2) \leq \Delta(G^2) + 1$, where $\omega(G^2)$ is the clique number and $\Delta(G^2)$ is the maximum vertex degree in G^2 .

3. Optimization Engine: Five Heuristics Plus Two Exact Solvers

The solver engine implements **five distinct heuristics** for fast, near-optimal scheduling, complemented by **two exact solvers** that prove global optimality:

3.1 Heuristic Algorithms

1. **Largest-Degree-First (LDF / Welsh-Powell)**: Sorts vertices descending by degree in G^2 ; colors bottlenecks first. $O(V \log V + E)$.
2. **DSATUR (Degree of Saturation)**: Dynamically selects uncolored node with highest count of distinct colored neighbors; tie-breaks by uncolored degree. $O(V^2 + E)$.
3. **Smallest-Last (Degeneracy / Matula-Beck)**: Successively eliminates minimum-degree nodes to bound colors by graph degeneracy. $O(V + E)$.
4. **Seeded Randomized Restarts**: Evaluates $N = 1000$ seeded permutations to escape greedy local minima deterministically.
5. **Local Search & Color Reduction**: Eliminates color classes via 2-color Kempe-chain swaps and min-conflicts Tabu search (up to 2000 iterations).

3.2 Exact Solvers

- A. **Google OR-Tools CP-SAT (Primary)**: 0-1 ILP constraint optimization with binary variables $x_{v,c}$ and slot indicators y_c . Symmetry breaking via maximum clique pre-coloring eliminates color permutation explosion.
- B. **Pure-Python Branch-and-Bound (Fallback)**: Zero-dependency backtracking solver with DSATUR variable branching, clique pre-coloring, and lower-bound pruning. Activates automatically when OR-Tools is not installed, or via `--force-pure-python-exact`.

3.3 Design Process & Decisions

Why graph squaring? The TDMA conflict constraints require that no two nodes within 2 hops share a slot. Rather than encoding hop-distance checks into every coloring algorithm, we reduce the problem to ordinary vertex coloring on the squared graph G^2 . This separation lets us reuse standard graph coloring theory and algorithms directly, simplifying both implementation and correctness proofs.

Why these five heuristics? Each targets a different structural weakness: LDF handles high-degree bottlenecks, DSATUR adapts dynamically to partial colorings, Smallest-Last exploits sparse substructure, randomized restarts escape greedy traps, and local search compresses post-hoc. Together they cover the spectrum from speed-optimized to quality-optimized.

What was tried and rejected? Pure Genetic Algorithms were prototyped but rejected due to excessive runtime (> 5 seconds), high stochastic variance, and poor constraint satisfaction. Simulated Annealing on unconstrained encodings frequently converged with 1-2 lingering conflicts. Naive ILP without symmetry breaking was orders of magnitude slower; adding maximum clique fixation resolved this.

Why an exact solver? Heuristics cannot prove optimality. Adding CP-SAT (and a zero-dependency fallback) lets us certify that the heuristic result is globally optimal on every benchmark topology.

Why a separate verifier? Algorithmic circularity is a common correctness trap: if the coloring code has a bug in G^2 construction, a verifier sharing that code would not catch it. The independent BFS-based verifier operates on raw physical graph G , providing a genuinely independent correctness check.

4. Empirical Benchmark Results

Topology Scenario	Nodes	G Edges	G ² Edges	Max Deg	Exact Opt	Best Heur	Gap	Runtime
1. 4x4 Grid (300 m)	16	42	90	15	9 slots	9 slots	0 (Opt)	0.48 ms
2. Sparse Linear (350 m)	16	15	29	4	3 slots	3 slots	0 (Opt)	0.17 ms
3. Dense Cluster (≤500 m)	16	120	120	15	16 slots	16 slots	0 (Opt)	0.22 ms
4. Two Disconnected Clusters	16	56	56	7	8 slots	8 slots	0 (Opt)	0.30 ms

4.1 Why the 4x4 Grid Requires 9 Slots

The assignment brief includes an illustrative sample showing '*Optimized Frame Length : 5 unique timeslots*'. The brief's sample shows 5 slots but does not give all coordinates, so it is treated as an illustrative format example, not a target. Our independent verifier is the correctness check.

Rigorous Mathematical Proof: In a 4x4 grid with 300 m spacing and 500 m radio range, diagonal nodes have Euclidean distance $\sqrt{300^2 + 300^2} \approx 424.26 \text{ m} \leq 500.0 \text{ m}$ (in direct range). Consequently, any 3x3 subgrid (9 nodes) has pairwise hop distance ≤ 2 in G , inducing a 9-node clique in G^2 : $\omega(G^2) \geq 9$. Because $\chi(G^2) \geq \omega(G^2)$, fewer than 9 slots mathematically violates Distance-1 or Distance-2 constraints. Our independent verifier confirms 9 slots is the exact optimal chromatic number.

5. Independent Verification Methodology

To guarantee algorithmic independence, `tdma.verify` does not share code with graph coloring or graph squaring routines. It executes BFS shortest-path queries directly on physical graph G , validating three mandatory invariants:

- **Node Coverage:** Every node in V has exactly one assigned timeslot.
- **Slot Contiguity:** Slot indices span $0 \dots K-1$ with zero gaps.
- **Conflict Freedom:** Any pair sharing a slot must satisfy $\text{dist}_G(u, v) \geq 3$. Flags direct (1-hop) and hidden-terminal (2-hop) collisions with offending neighbor sets.

6. Part 2: EMANE Integration & Official Verification Log

The EMANE bridge integrates optimization output with Adjacent Link's official TDMA radio model (tdmaeventschedulerradiomodel). Configured with 1 ms slots (1000 μ s), 50 μ s guard overhead, and 2.4 GHz carrier frequency.

6.1 Architecture & Configuration

- **MAC Model:** tdmaeventschedulerradiomodel — timing parameters (slotduration, slotoverhead, bandwidth) are attributes of the <structure> XML element, not MAC-level parameters.
- **PHY Configuration:** 2.4 GHz carrier, 20 MHz bandwidth, 0 dBm transmit power.
- **Schedule XML:** Root <emane-tdma-schedule> containing <structure>, <multiframe>, <frame>, and <slot> elements with <tx>, <rx>, or <idle> children.

6.2 Tested vs. Design-Only Scope

- **Tested Offline:** Automated JSON-to-XML translation, NEM mapping, schedule matrix validation, round-trip schema parsing, and Python publisher generation.
- **Design-Only:** Live over-the-air RF packet exchange inside Linux kernel network stack (requires root and TAP device).

6.3 Official EMANE Verification Log

Parameter / Component	Category	Verified Name / Setting	Official URL & Quoted Line	Status
MAC Model Library	MAC Plugin	tdmaeventschedulerradiomodel	tdmaradiomodel.xml.in: <mac library='tdmaeventschedulerradiomodel'>	VERIFIED
MAC vs Structure	Architecture	<structure> holds timing	tdma-radio-model.txt: 'The TDMA structure defines: Slot size..., Slot overhead..., slots per frame...'	VERIFIED
MAC PCR Curve URI	MAC Param	pcrcurveuri	tdmaradiomodel.xml.in: <param name='pcrcurveuri' value='...tdmabasemodelpcr.xml'/>	VERIFIED
MAC Queue Controls	MAC Param	queue.depth, aggregation, etc.	tdmaradiomodel.xml.in: <param name='queue.depth' value='255'/>	VERIFIED
Schedule Root Element	XML Schema	<emane-tdma-schedule>	tdmaschedule.xsd (line 71): <xs:element name='emane-tdma-schedule'>	VERIFIED
Structure Element	XML Element	<structure frames=.. slots=..>	tdmaschedule.xsd: <xs:element name='structure' slotduration=.. slotoverhead=..>	VERIFIED
Multiframe / Frame	XML Element	<multiframe> / <frame>	tdmaschedule.xsd: <xs:element name='multiframe' containing <frame>	VERIFIED
Slot Allocation	XML Element	<slot index=.. nodes=..><tx/>	tdmaschedule.xsd: <xs:element name='slot' with <tx>, <rx>, <idle>	VERIFIED
CLI Injection Tool	CLI Script	emaneevent-tdmaschedule	tdma-radio-model.txt (line 376): \$ emaneevent-tdmaschedule schedule.xml -i lo	VERIFIED
Python Event Class	Python API	emane.events.TDMAScheduleEvent	tdmascheduleevent.py: class TDMAScheduleEvent; def structure(..); def append(..)	VERIFIED
Python Event Service	Python API	emane.events.EventService	eventservice.py: class EventService(eventchannel); def publish(nemId, event)	VERIFIED

6.4 Radio Propagation & Range Modeling (Design Only / Untested)

EMANE's TDMA scheduler radio model does not have built-in knowledge of the discrete 500 m communication range limit. Which virtual radios hear each other is determined strictly by node locations and RF pathloss (LocationEvent [ID 100] / PathlossEvent [ID 101]) combined with receiver sensitivity and the propagation model (freespace / 2ray). To demonstrate spatial reuse in emulation, virtual radios must be placed at the exact coordinates of the input topology and pathloss calibrated so the effective range is ~500 m; otherwise all nodes hear each other globally and concurrent transmissions (e.g. Node_01 and Node_04 sharing Slot 8) would collide. *Status: DESIGN ONLY / UNTESTED. Schedule XML generation and schema parity are fully tested offline; live RF propagation tuning and packet-level slot enforcement have not been executed on a live Linux kernel testbed.*

7. Assumptions & Edge-Case Handling

- **Exact 500.0 m Boundary:** Distances are computed via Euclidean metric $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ with tolerance 10^{-9} m. Exactly 500.0 m is strictly in-range.
- **Duplicate Coordinates:** Reject co-located nodes with exit code 2 and explicit error output.
- **Slot Compaction:** Active color sets are relabeled contiguously to $0 \dots K-1$.
- **Arbitrary Topologies ($n \geq 1$):** Gracefully handles isolated nodes, single nodes, disconnected graphs, and arbitrary scales.

8. Future Roadmap

1. **Multi-Channel TDMA:** 2D time-frequency coloring (t, f) across orthogonal carrier channels.
2. **Dynamic Mobile MANET:** Distributed reservation protocols (DRAND / C-TDMA) for moving topologies.
3. **Directed Link Scheduling (STDMA):** Link-based spatial reuse allowing concurrent transmission to disjoint receivers.